

MAINTENANCE

Software Maintenance

Metodologia di sviluppo

Per lo sviluppo del sistema è stato adottato un approccio di Forward Engineering seguendo tali fasi:

- Fase iniziale: a partire dai diagrammi (prodotti con Papyrus), il team ha costruito lo scheletro del codice ed ha analizzato le funzionalità da implementare nel sistema
- Implementazione: il team ha lavorato sul codice in maniera manuale integrando; le logiche di business con il framework Vaadin e il database Firebase.
- Manutenzione: sono state svolte attività di refactoring per ottenere un codice più pulito, leggibile e con una facile manutenzione.

Attività di refactoring effettuate

Il processo di refactoring è stato fondamentale per rispettare il requisito di **Manutenibilità**. Di seguito vengono descritti i principali interventi effettuati:

1. Estrazione della logica di business dalle Viste (Extract class):

- Prima: Molte logiche di interazione con il database erano cablate direttamente all'interno delle classi UI (es. LoginView o StationsView).
- Dopo: La logica è stata assegnata a classi di servizio dedicate come UtentiService, ColonnineService e PrenotazioniService. Questo permette di modificare la logica di business senza dover toccare l'interfaccia grafica.

2. Pulizia del codice ed eliminazione di codice morto o non utilizzato:

Intervento: Durante la fase finale del progetto, sono stati rimossi metodi di test inutilizzati, variabili locali non referenziate e import ridondanti. È stata inoltre rimossa la gestione manuale di alcuni file frontend ora gestiti automaticamente dal bundle di Vaadin.

3. Divisione di classi complesse in sottoclassi (es Firebase) :

- Prima: Esisteva un unico punto di accesso generico per tutte le chiamate al database.
- Dopo: La gestione è stata suddivisa per competenza in classi specifiche come FirebaseUtentiService, FirebaseColonnineService e FirebasePrenotazioniService. Ogni classe implementa una specifica interfaccia (es. UtentiInterface) garantendo il disaccoppiamento.

4. Controlli unificati in un'unica classe di validazione:

- Prima: I controlli sulla validità delle email o delle password erano sparsi tra RegisterView e altre classi di input.
- Dopo: Tutti i criteri di validazione sono stati centralizzati nella classe DataValidator. Gestisce in modo univoco la verifica per le password, delle email e la correttezza degli slot di prenotazione.

5. Divisione di classi complesse in metodi:

I metodi eccessivamente lunghi (specialmente nel salvataggio dei dati asincrono) sono stati suddivisi in sotto-metodi privati. Ad esempio, in UtentiService, la logica di registrazione è stata separata dalla logica di invio email di benvenuto tramite l'uso di CompletableFuture.

6. Divisione precisa tra parte java e parte CSS:

La gestione dello stile è stata delegata al file CSS esterni o annotazioni @CssImport, evitando di definire stili "inline" all'interno delle classi Java. In questo modo c'è stato un miglioramento sia per la pulizia del codice e sia per la separazione dei compiti tra frontend e backend.

Utilizzo di reverse engineering

Sebbene il progetto abbia seguito principalmente un approccio di forward engineering, durante la fase di realizzazione dei diagrammi è stata eseguita un'attività di reverse engineering.

L'obiettivo di questa fase è stato quello di aggiornare correttamente e coerentemente il diagramma delle classi con il codice all'ultima versione, migliorando e rendendolo più attinente al codice stesso.

In questo modo abbiamo ottenuto un diagramma molto più complesso, più ricco e più professionale.

Gestione delle Criticità

Il sistema è stato aggiornato per risolvere problemi di sincronizzazione e concorrenza del database. Con l'implementazione di controlli per gli accessi simultanei si è riusciti a garantire aggiornamenti corretti senza conflitti tra utenti diversi. Un esempio è la gestione dello stato di prenotazione delle colonnine elettriche o lo stato di carica dell'auto.